

Operations Portal — Runbook

Every command in order: local development, tests, AWS provisioning, teardown.

Repository	Gupta0um/erp-crm-operations-portal	AWS account	758530010955
Region	ap-south-1	Terraform	1.16.0

1. Local development

The API and the frontend run as two Node processes against a hosted PostgreSQL. Containers are not part of the local loop; the image build happens in the deploy pipeline and lands in ECR at stage 3.

1. Point the API at a database

This machine has no local PostgreSQL, so use the Supabase session pooler and give the run its own schema.

```
cd backend
cp .env.example .env
```

Port 5432, not 6543.

The session pooler on 5432 can run migrations; the transaction pooler on 6543 cannot. Append ? schema=erp_local so this run never touches the live erp_portal data.

2. Install, migrate, seed, run — API on port 4000

```
npm install
npx prisma migrate deploy
npm run seed
npm run dev
```

3. Start the frontend in a second terminal — port 5173

Vite proxies /api to port 4000, so no CORS configuration is needed in development.

```
cd frontend
npm install
npm run dev
```

4. Sign in

Open <http://localhost:5173> and use `admin@example.com` with `Portal@2026`.

2. Tests

Vitest drives the real Express app through Supertest against a real database. Every table is truncated between tests, so the target must be disposable.

```
cd backend
cp .env.test.example .env.test # DATABASE_URL must point at a throwaway schema
npm test
npm run test:coverage          # optional, writes text and lcov
```

Truncation is unconditional. Never point `.env.test` at the live database. With no local PostgreSQL, let CI run the suite instead: it uses a `postgres:16` service container and currently passes 218 tests across 8 files.

```
gh run list --workflow CI --limit 1
gh run watch
```

3. AWS provisioning, stage 3

From an empty account to a load balanced, auto scaled build behind CloudFront. Roughly 25 minutes cold. Every rebuild issues a new CloudFront domain, so read the output rather than reusing an old URL.

1. Check the tools answer

```
terraform version
aws sts get-caller-identity
gh auth status
```

Do not run `winget upgrade Hashicorp.Terraform`.

The package ships 1.15.8, which refuses to read state written by 1.16.0. The binary was replaced by hand; an upgrade would revert it and break teardown.

2. Raise the infrastructure

Runs `terraform init`, applies with `stage=3`, then prints the outputs. Stage 1 and stage 2 use the same script with a different number.

```
./scripts/up.sh 3
```

3. Turn the deploy workflow back on

It is set to `false` while AWS is torn down, so the job skips instead of failing on a role that does not exist.

```
gh variable set DEPLOY_ENABLED -b "true"
```

4. Publish the first release

Pushes the API image to ECR and the React build to S3 over OIDC, with no stored AWS keys.

```
gh workflow run deploy.yml
gh run watch
```

5. Scale the group to two instances

The group starts at one so the first image has somewhere to land. This step is what makes instance loss survivable.

```
terraform -chdir=infra apply -auto-approve -var stage=3 -var asg_min_size=2
```

6. Read the URL and verify

```
terraform -chdir=infra output
curl -s https://<cloudfront-domain>/api/health
```

A `{"status":"ok"}` reply means the balancer, the instances and the database are all answering.

Proving the resilience claims on camera

CLAIM	COMMAND	EXPECTED
Database failover	<code>aws rds reboot-db-instance --force-failover --db-instance-identifier erp-portal-db</code>	primary moves zone, API recovers itself in about 25s
Instance loss	<code>aws autoscaling terminate-instance-in-auto-scaling-group --should-decrement-desired-capacity false --instance-id <id></code>	one failed request, healthy again in about 2m30s

4. Teardown

Run this the moment the demo ends. The NAT gateway, the load balancer and RDS are the three that cost real money per hour.

```
./scripts/down.sh
gh variable set DEPLOY_ENABLED -b "false"
```

A non-zero exit is not always a failure. `down.sh` checks unattached Elastic IPs across the whole account, so it reports a leftover while an unrelated idle IP from another project still exists. Read the per-resource lines: NAT gateways, load balancers, RDS instances and Auto Scaling groups must all say *clear*.

What stays up regardless

SERVICE	URL	COST
Portal, Render	<code>erp-portal-web.onrender.com</code>	free tier
API, Render	<code>erp-portal-api.onrender.com/api</code>	free tier, sleeps after ~15 min idle
Database, Supabase	<code>erp_portal schema</code>	free tier

Warm the API before recording. The first request after a sleep takes up to a minute. One `curl -s https://erp-portal-api.onrender.com/api/health` a few seconds ahead of the take is enough.

Demo accounts: `admin@`, `sales@`, `warehouse@`, `accounts@example.com` — all with the password `Portal@2026`.